Roots

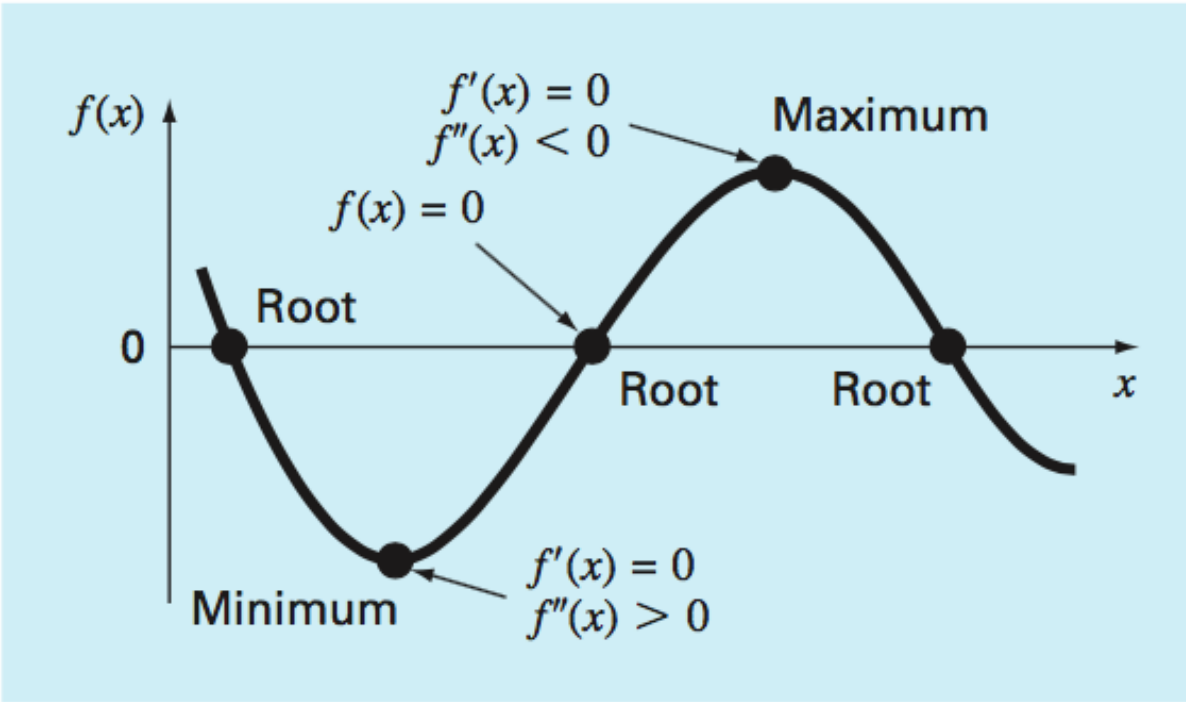


FIGURE PT2.1
A function of a single variable illustrating the difference between roots and optima.

Overview

- Roots
  - Points of a nonlinear equation where $f(x) = 0$
  - **Bracketing methods**: start with a guess for a range containing the root and reduce the width of the bracket iteratively. (reliable)
    - Bisection method
    - False position method
  - **Open Methods** (fast)
    - Fixed-point iteration
    - Newton-Raphson
    - Secant methods
  - Hybrid Methods: Brent's root finding method (Fast and reliable)
  - MATLAB: fzero function

Overview

- Optimization
  - Bracketing methods:
    - Golden-section search
    - Parabolic interpolation
  - Hybrid approach: combine golden-section and quadratic interpolation
  - MATLAB: fminbnd
  - Multidimensional optimization
    - MATLAB: fminsearch

Motivation

- How to Solve?

$$f(x) = ax^2 + bx + c = 0$$

Motivation

- How to Solve?

$$f(x) = ax^2 + bx + c = 0$$

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- One method to obtain an approximate solution is to plot the function and determine where it crosses the *x* axis.

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- One method to obtain an approximate solution is to plot the function and determine where it crosses the *x* axis.
-

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- One method to obtain an approximate solution is to plot the function and determine where it crosses the *x* axis.
- Although graphical methods are useful for obtaining rough estimates of roots, they are limited because of their lack of precision.

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- An alternative approach is to use trial and error.
- This “technique” consists of guessing a value of *x* and evaluating whether *f*(*x*) is zero.

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- An alternative approach is to use trial and error.
- This “technique” consists of guessing a value of *x* and evaluating whether *f*(*x*) is zero.
- If not (as is almost always the case), another guess is made, and ***f*(*x*) is again evaluated** to determine whether the new value provides a better estimate of the root. **The process is repeated until a guess is obtained** that results in an *f*(*x*) that is close to zero.

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- An alternative approach is to use trial and error.
- This “technique” consists of guessing a value of *x* and evaluating whether *f*(*x*) is zero.
- If not (as is almost always the case), another guess is made, and ***f*(*x*) is again evaluated** to determine whether the new value provides a better estimate of the root. **The process is repeated until a guess is obtained** that results in an *f*(*x*) that is close to zero.
- Such methods may be inadequate!

Motivation

- How to Solve? Find x that makes the function f(x) zero (find root).

$$f(x) = e^{-x} - x$$

Cannot be solved analytically.

- An alternative approach is to use trial and error.
- This “technique” consists of guessing a value of *x* and evaluating whether *f*(*x*) is zero.
- If not (as is almost always the case), another guess is made, and ***f*(*x*) is again evaluated** to determine whether the new value provides a better estimate of the root. **The process is repeated until a guess is obtained** that results in an *f*(*x*) that is close to zero.
- Such methods may be inadequate!
- Thus, we will learn **systematic approaches** to approximate true value of root.

Motivation

$$v = \frac{gm}{c}\left(1 - e^{-(c/m)t}\right)$$

- where velocity *v* the dependent variable, time *t* the independent variable, the gravitational constant *g* the forcing function, and the drag coefficient *c* and mass *m* parameters.

Motivation

$$v = \frac{gm}{c}\left(1 - e^{-(c/m)t}\right)$$

- where velocity *v* the dependent variable, time *t* the independent variable, the gravitational constant *g* the forcing function, and the drag coefficient *c* and mass *m* parameters.
- it cannot be solved explicitly for the drag coefficient.
- There is no way to rearrange the equation so that *c* is isolated on one side of the equal sign.

Motivation

$$v = \frac{gm}{c}\left(1 - e^{-(c/m)t}\right)$$

- where velocity *v* the dependent variable, time *t* the independent variable, the gravitational constant *g* the forcing function, and the drag coefficient *c* and mass *m* parameters.
- it cannot be solved explicitly for the drag coefficient.
- There is no way to rearrange the equation so that *c* is isolated on one side of the equal sign.
- In such cases, *c* is said to be *implicit.*

Motivation

$$v = \frac{gm}{c}\left(1 - e^{-(c/m)t}\right)$$

- To solve the problem using numerical methods, it is conventional to reexpress Eq above.

$$f(c) = \frac{gm}{c}\left(1 - e^{-(c/m)t}\right) - v$$

- The value of *c* that makes *f*(*c*) 0 is, therefore, the root of the equation.

Mathematical Bckground.

- By definition, a function given by $y = f(x)$ is algebraic if it can be expressed in the form:

$$f_n y^n + f_{n-1} y^{n-1} + \cdots + f_1 y + f_0 = 0$$

- *Polynomials* are a simple class of algebraic functions that are represented generally by

$$f_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

- Some examples:
-

$$f_2(x) = 1 - 2.37x + 7.5x^2$$

$$f_6(x) = 5x^2 - x^3 + 7x^6$$

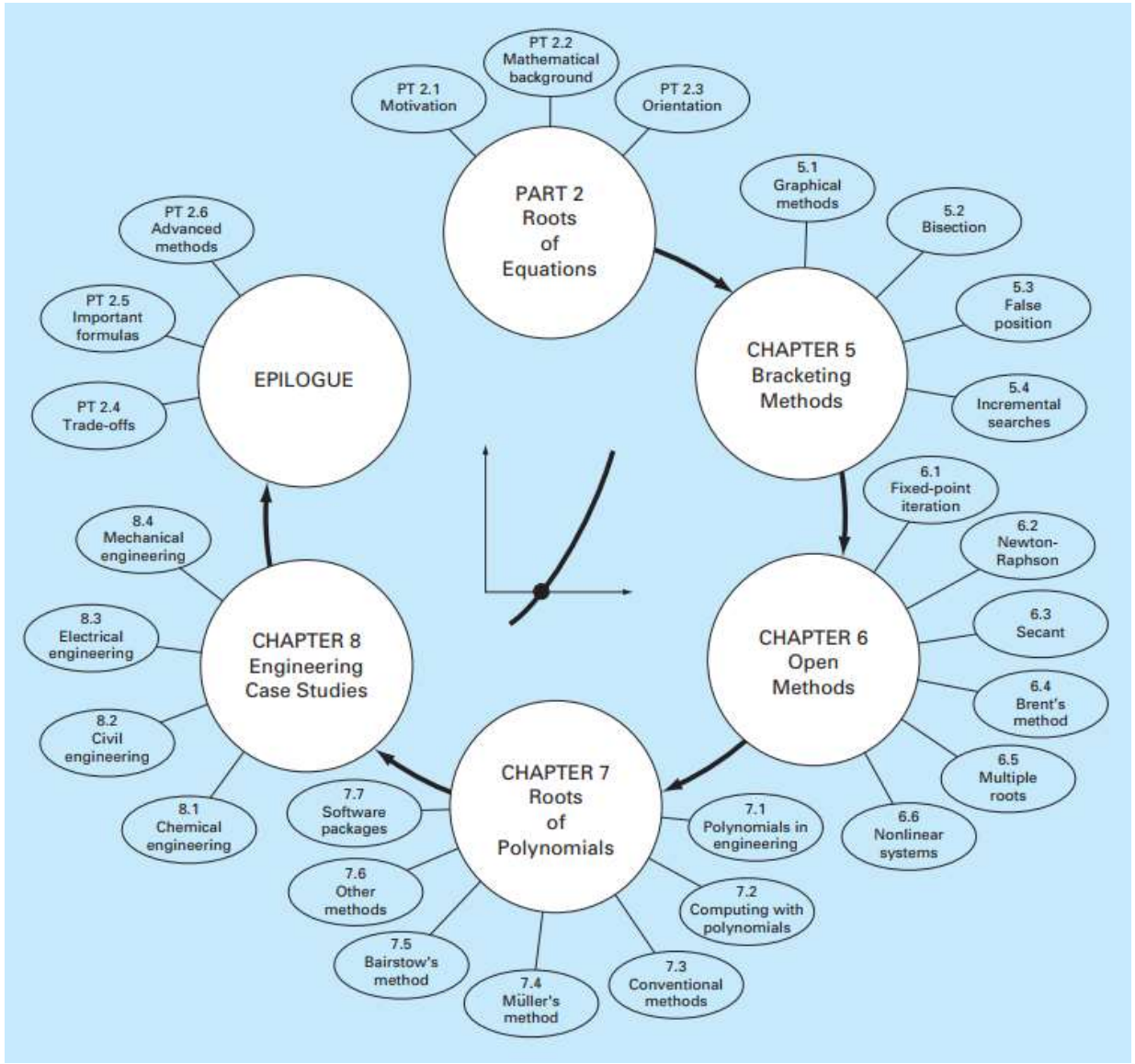
Mathematical Bckground.

- A *transcendental* function is one that is nonalgebraic. These include trigonometric,exponential, logarithmic, and other, less familiar, functions.
- Some examples:

$$f(x) = \ln x^2 - 1$$

$$f(x) = e^{-0.2x}\sin(3x - 0.5)$$

Orientation.



Roots: Bracketing Methods

Chapter Objectives

- Understanding what roots problems are and where they occur in engineering and science.
- Knowing how to determine a root graphically.
- Understanding the incremental search method and its shortcomings.
- Knowing how to solve a roots problem with the **bisection method**.
- Knowing how to estimate the error of bisection and why it differs from error estimates for other types of root location algorithms.
- Understanding **false position** and how it differs from bisection.

Graphical Methods

Problem Statement. Use the graphical approach to determine the drag coefficient c needed for a parachutist of mass $m = 68.1$ kg to have a velocity of 40 m/s after free-falling for time $t = 10$ s. *Note:* The acceleration due to gravity is 9.81 m/s^2.

Solution. This problem can be solved by determining the root of Eq. (PT2.4) using the parameters $t = 10$, $g = 9.81$, $v = 40$, and $m = 68.1$:

$$f(c) = \frac{9.81(68.1)}{c}(1 - e^{-(c/68.1)10}) - 40$$

- or

$$f(c) = \frac{668.06}{c}(1 - e^{-0.146843c}) - 40$$

Graphical Methods

Problem Statement. Use the graphical approach to determine the drag coefficient c needed for a parachutist of mass $m = 68.1$ kg to have a velocity of 40 m/s after free-falling for time $t = 10$ s. *Note:* The acceleration due to gravity is 9.81 m/s^2.

Solution. This problem can be solved by determining the root of Eq. (PT2.4) using the parameters $t = 10$, $g = 9.81$, $v = 40$, and $m = 68.1$:

$$f(c) = \frac{9.81(68.1)}{c}(1 - e^{-(c/68.1)10}) - 40$$

- or

$$f(c) = \frac{668.06}{c}(1 - e^{-0.146843c}) - 40$$

c	f(c)
4	34.190
8	17.712
12	6.114
16	−2.230
20	−8.368

Graphical Methods

Problem Statement. Use the graphical approach to determine the drag coefficient c needed for a parachutist of mass $m = 68.1$ kg to have a velocity of 40 m/s after free-falling for time $t = 10$ s. *Note:* The acceleration due to gravity is 9.81 m/s^2.

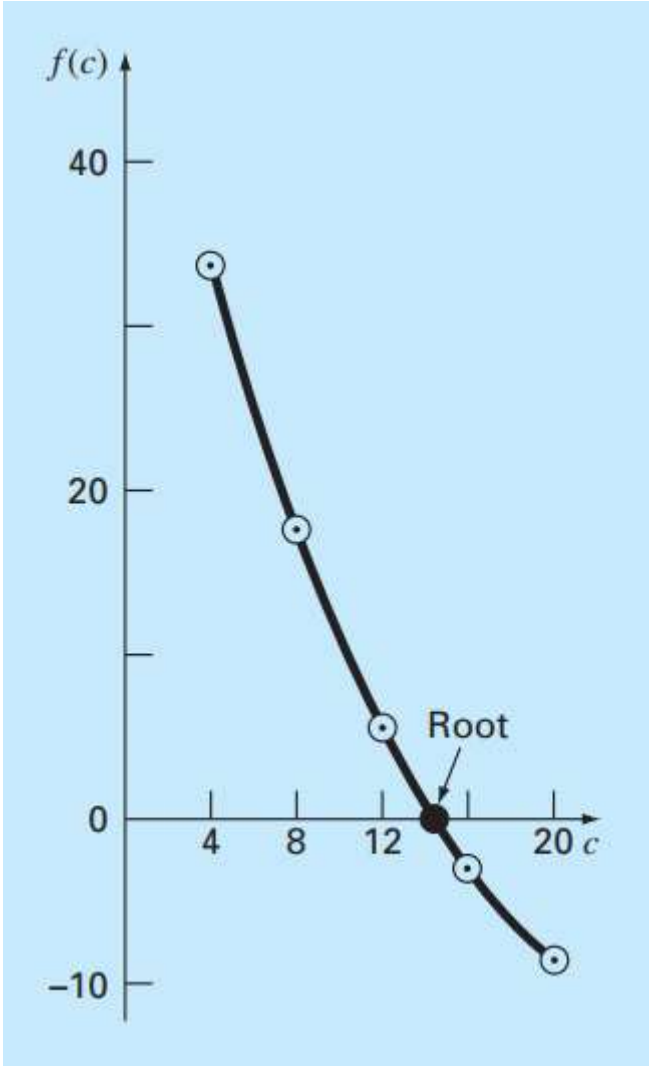
Solution. This problem can be solved by determining the root of Eq. (PT2.4) using the parameters $t = 10$, $g = 9.81$, $v = 40$, and $m = 68.1$:

$$f(c) = \frac{9.81(68.1)}{c}(1 - e^{-(c/68.1)10}) - 40$$

- or

$$f(c) = \frac{668.06}{c}(1 - e^{-0.146843c}) - 40$$

c	$f(c)$
4	34.190
8	17.712
12	6.114
16	−2.230
20	−8.368



Graphical Methods

Problem Statement. Use the graphical approach to determine the drag coefficient c needed for a parachutist of mass $m = 68.1$ kg to have a velocity of 40 m/s after free-falling for time $t = 10$ s. *Note:* The acceleration due to gravity is 9.81 m/s^2.

Solution. This problem can be solved by determining the root of Eq. (PT2.4) using the parameters $t = 10$, $g = 9.81$, $v = 40$, and $m = 68.1$:

$$f(c) = \frac{9.81(68.1)}{c}(1 - e^{-(c/68.1)10}) - 40$$

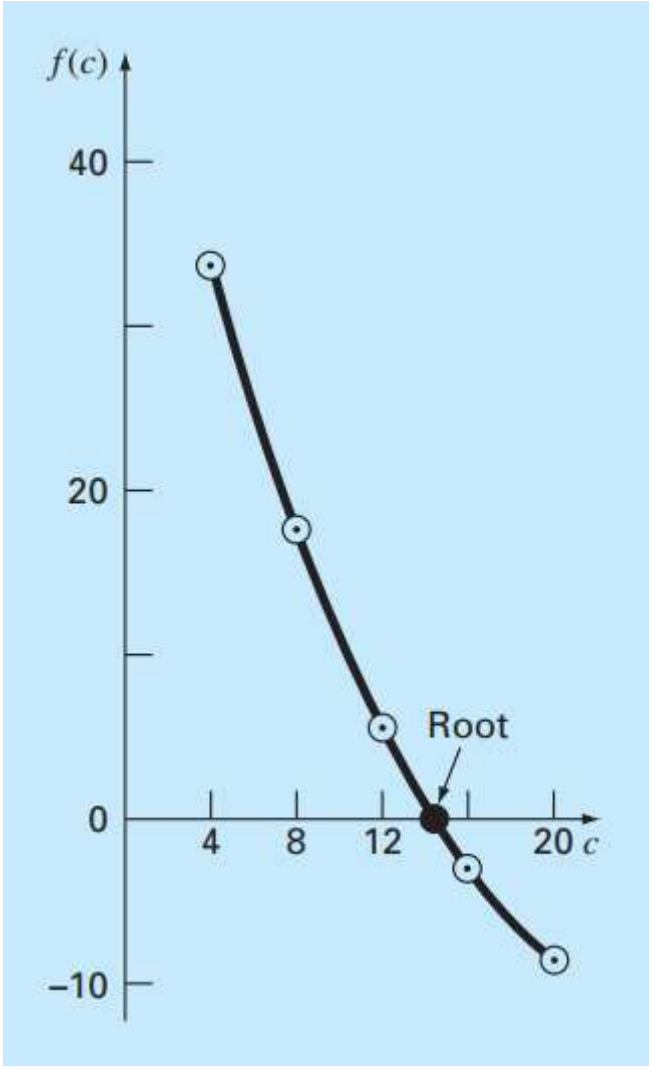
$$f(c) = \frac{668.06}{c}(1 - e^{-0.146843c}) - 40$$

Visual inspection of the plot provides a rough estimate of the root of 14.75.

c	f(c)
4	34.190
8	17.712
12	6.114
16	−2.230
20	−8.368

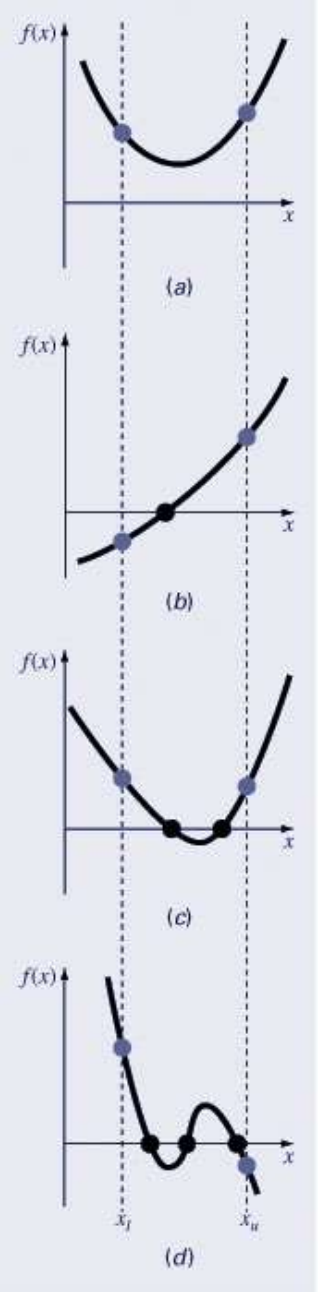
$$v = \frac{9.81(68.1)}{14.75}(1 - e^{-(14.75/68.1)10}) = 40.100$$

which is very close to the desired fall velocity of 40 m/s.

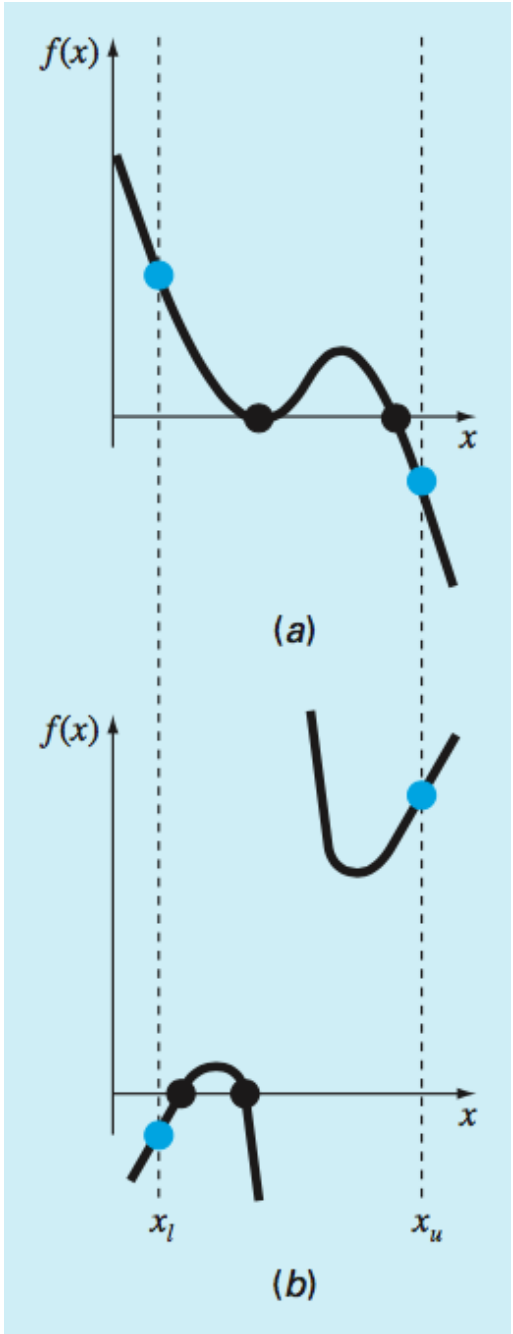


Graphical Methods

- A simple method for obtaining the estimate of the root of the equation *f(x)*=0 is to make a plot of the function and observe where it crosses the *x*-axis.
- Graphing the function can also indicate where roots may be and where some root-finding methods may fail:
  a) Same sign, no roots
  b) Different sign, one root
  c) Same sign, two roots (**even number of roots**)
  d) Different sign, three roots (**odd number of roots**)



Exceptions



The two major classes of methods available are distinguished by the type of initial guess. They are

- *Bracketing methods*. As the name implies, these are based on two initial guesses that "bracket" the root—that is, are on either side of the root.
- *Open methods*. These methods can involve one or more initial guesses, but there is no need for them to bracket the root.

Roots

- Bracketing Methods:
  - Always work
  - Converge slowly
- Open Methods
  - Do not always work (can diverge)
  - Converge quicker

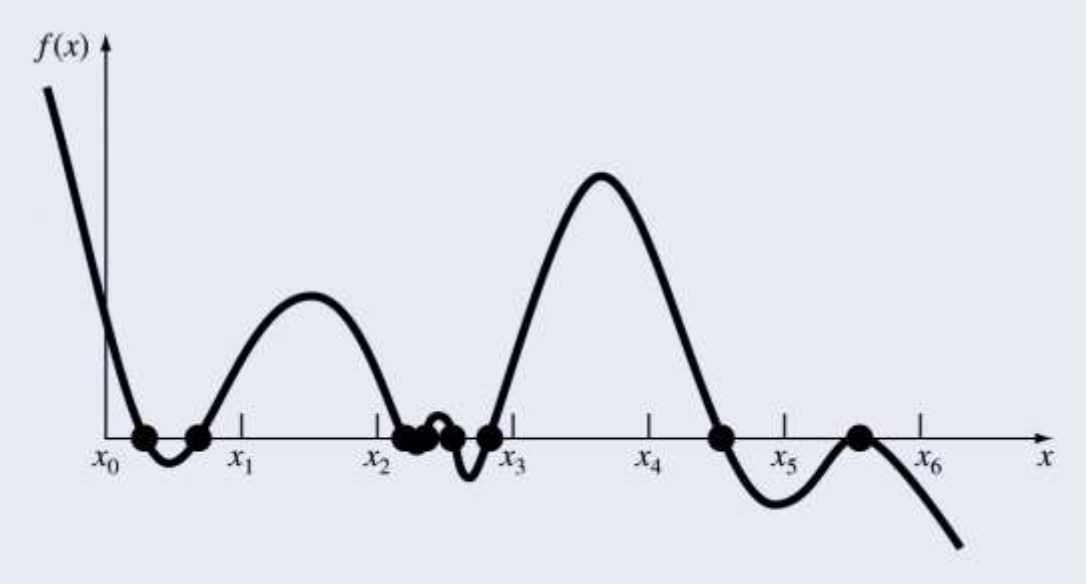
Incremental search

- Approach to obtain initial guesses
- The *incremental search* method tests the value of the function at evenly spaced intervals and finds brackets by identifying function sign changes between neighboring points.
- If $f(x_l) f(x_u) < 0$

then there is at least one real root between x_l and x_u.

Incremental Search Hazards

- If the spacing between the points of an incremental search are too far apart, brackets may be missed due to capturing an even number of roots within two points.
- Incremental searches cannot find brackets containing even-multiplicity roots regardless of spacing.



Cases where roots could be missed because the incremental length of the search procedure is too large. Note that the last root on the right is multiple and would be missed regardless of the increment length.

```
function xb = incsearch(func,xmin,xmax,ns)
% incsearch: incremental search root locator
%   xb = incsearch(func,xmin,xmax,ns):
%      finds brackets of x that contain sign changes
%      of a function on an interval
% input:
%   func = name of function
%   xmin, xmax = endpoints of interval
%   ns = number of subintervals (default = 50)
% output:
%   xb(k,1) is the lower bound of the kth sign change
%   xb(k,2) is the upper bound of the kth sign change
%   If no brackets found, xb = [].
if nargin < 3, error('at least 3 arguments required'), end
if nargin < 4, ns = 50; end %if ns blank set to 50
% Incremental search
x = linspace(xmin,xmax,ns);
f = func(x);
nb = 0; xb = []; %xb is null unless sign change detected
for k = 1:length(x)-1
  if sign(f(k)) ~= sign(f(k+1)) %check for sign change
    nb = nb + 1;
    xb(nb,1) = x(k);
    xb(nb,2) = x(k+1);
  end
end
if isempty(xb)    %display that no brackets were found
  disp('no brackets found')
  disp('check interval or increase ns')
else
  disp('number of brackets:') %display number of brackets
  disp(nb)
end
```

Example

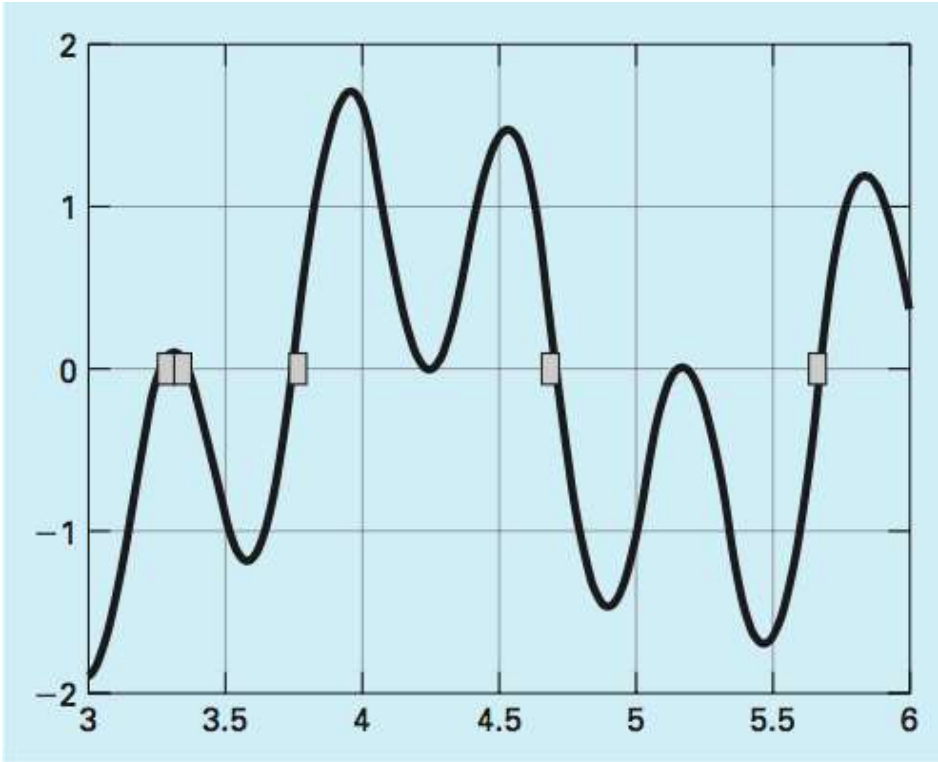
Problem Statement. Use the M-file `incsearch` (Fig. 5.4) to identify brackets within the interval [3, 6] for the function:

$$f(x) = \sin(10x) + \cos(3x)$$

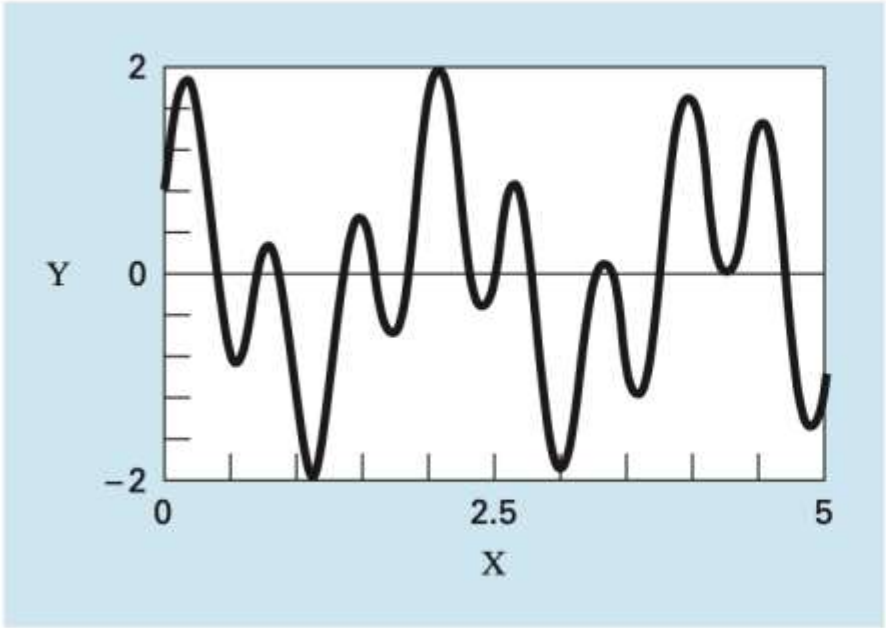
```
>> incsearch(@(x) sin(10*x)+cos(3*x),3,6)
number of brackets:
     5

ans =

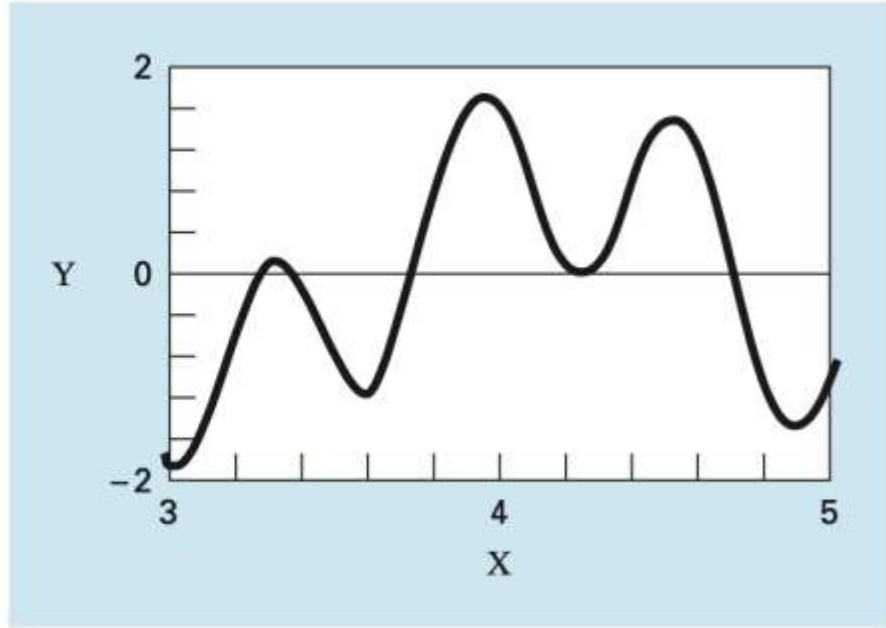
    3.2449    3.3061
    3.3061    3.3673
    3.7347    3.7959
    4.6531    4.7143
    5.6327    5.6939
```



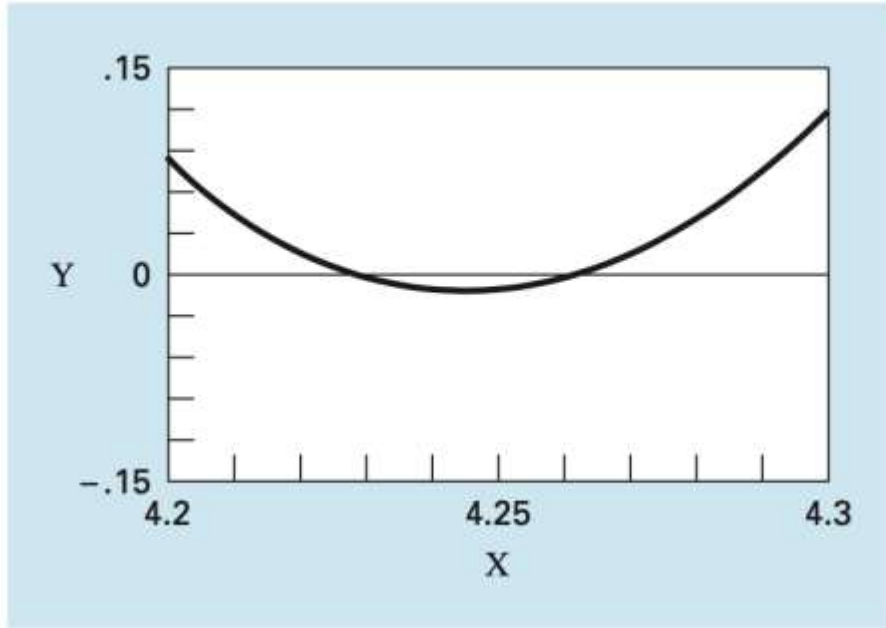
misses possible roots at $x \cong 4.25$ and 5.2.



(*a*)



(*b*)



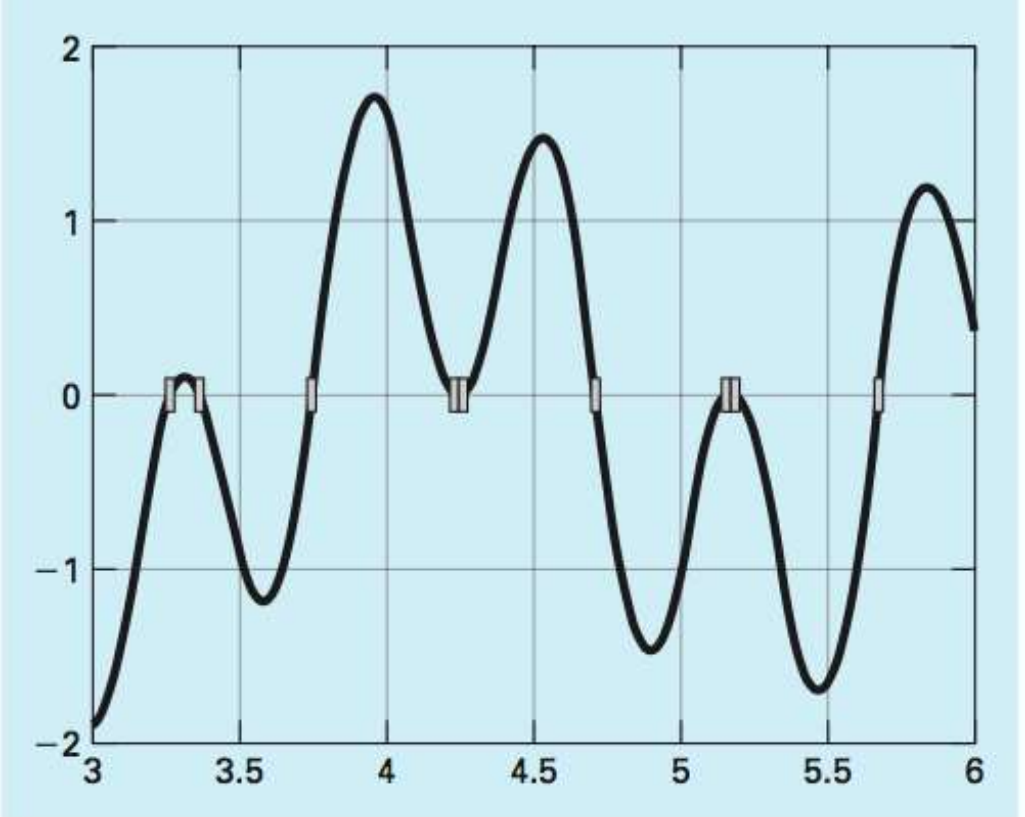
Example

- Run with more subintervals

```
>> incsearch(@(x) sin(10*x)+cos(3*x),3,6,100)

number of brackets:
     9

ans =
    3.2424    3.2727
    3.3636    3.3939
    3.7273    3.7576
    4.2121    4.2424
    4.2424    4.2727
    4.6970    4.7273
    5.1515    5.1818
    5.1818    5.2121
    5.6667    5.6970
```

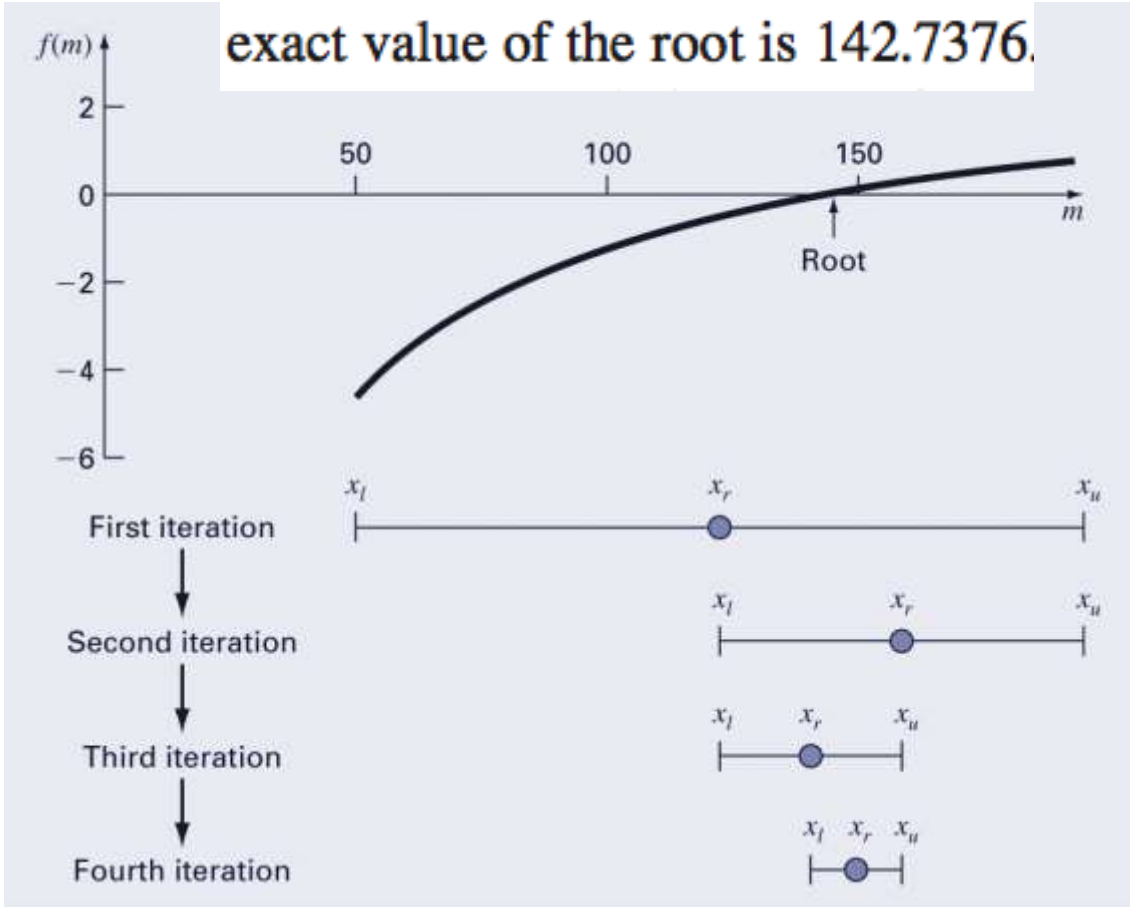


Bisection Method

Bisection Method

- The *bisection method* is a variation of the incremental search method in which **the interval is always divided in half.**
- If a function changes sign over an interval, the function value at the midpoint is evaluated.
- The location of the root is then determined as lying within the subinterval where the sign change occurs.
- The absolute error is reduced by a factor of 2 for each iteration.

exact value of the root is 142.7376.



function changes sign between values of 50 and 200.

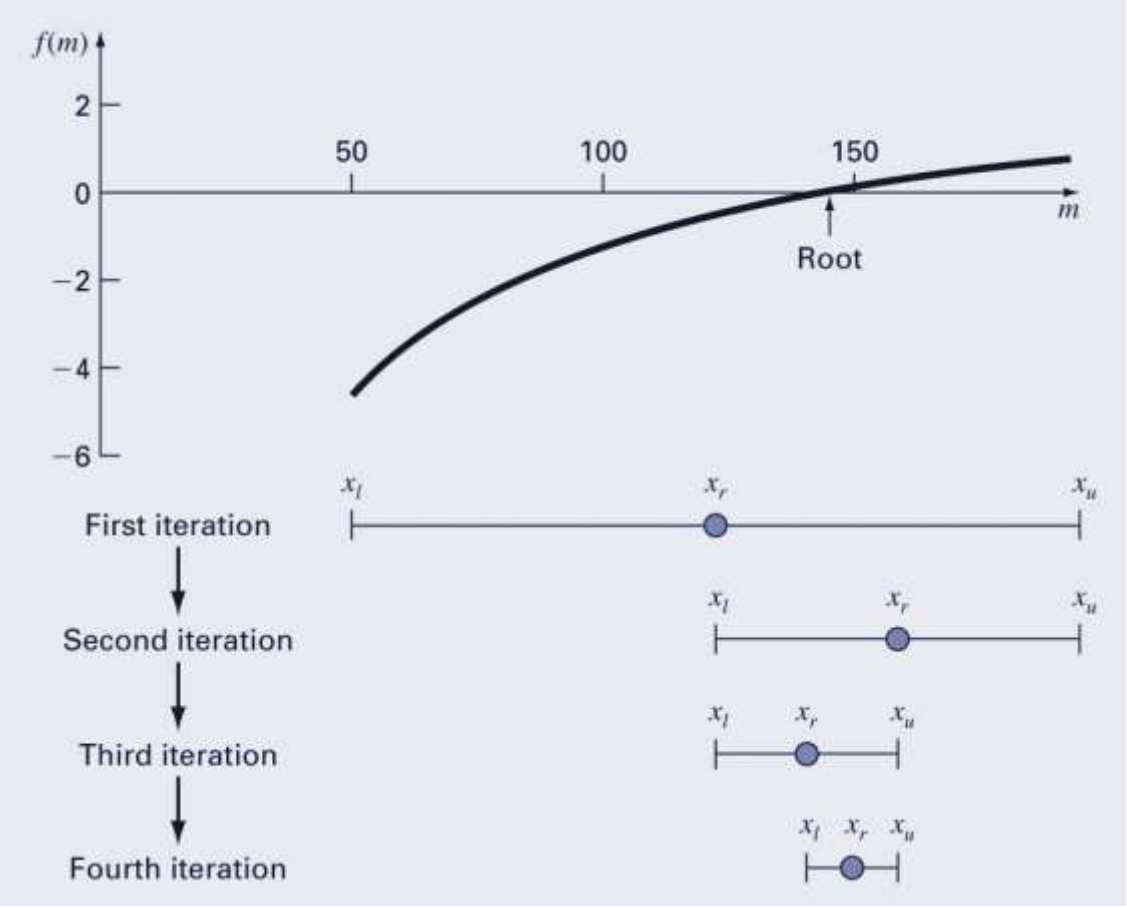
$$x_r = \frac{50 + 200}{2} = 125 \qquad x_l = 125 \text{ to } x_u = 200$$

$$x_r = \frac{125 + 200}{2} = 162.5$$

125 and 162.5.

Bisection Method

exact value of the root is 142.7376.



function changes sign between values of 50 and 200.

$$x_r = \frac{50 + 200}{2} = 125 \qquad x_l = 125 \text{ to } x_u = 200$$

$$|\varepsilon_t| = \left|\frac{142.7376 - 125}{142.7376}\right| \times 100\% = 12.43\%$$

$$x_r = \frac{125 + 200}{2} = 162.5 \qquad 125 \text{ and } 162.5.$$

$$|\varepsilon_t| = 13.85\%$$

$$x_r = \frac{125 + 162.5}{2} = 143.75$$

$$\varepsilon_t = 0.709\%$$

Bisection Method

- Iterate until the result is accurate enough.
- We need an error estimate that does not require the knowledge of the true root.
- Stop when the approximate relative percent error is below some value eps.
- Approximate relative percent error:

$$|\varepsilon_a| = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| 100\%$$

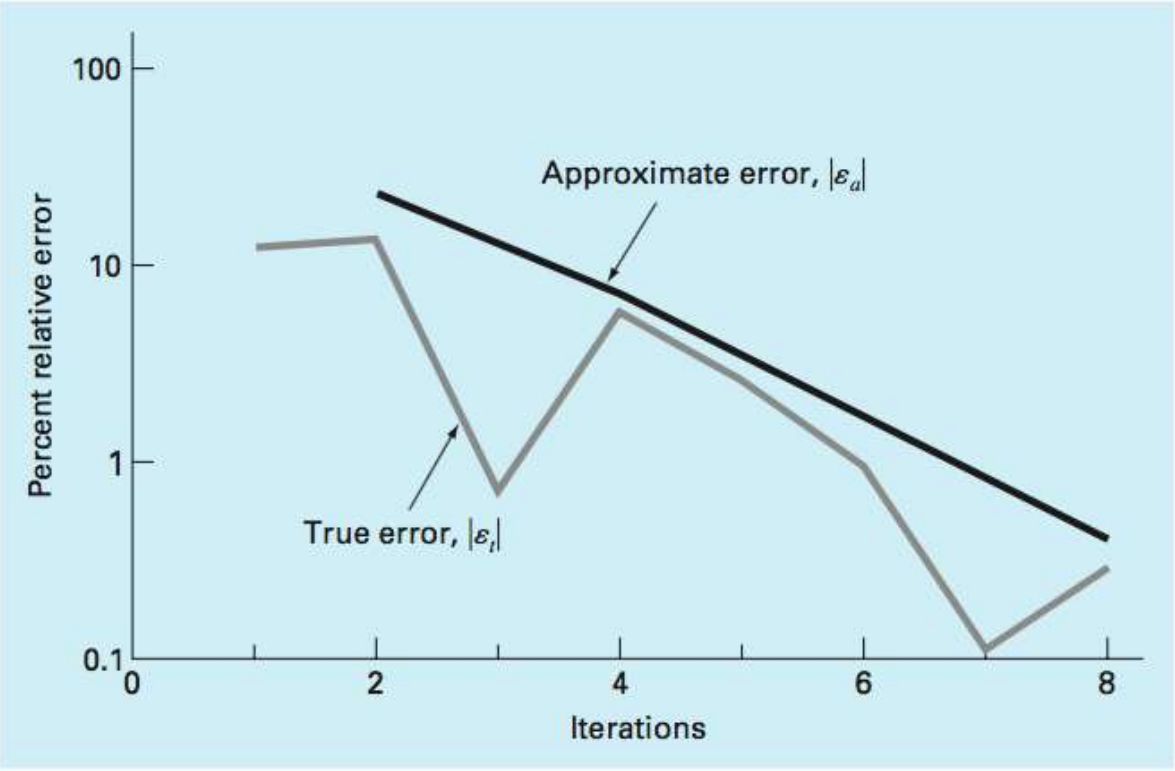
Bisection Method

Iteration	x_l	x_u	x_r	$\lvert\varepsilon_a\rvert$ (%)	$\lvert\varepsilon_t\rvert$ (%)
1	50	200	125		12.43
2	125	200	162.5	23.08	13.85
3	125	162.5	143.75	13.04	0.71
4	125	143.75	134.375	6.98	5.86
5	134.375	143.75	139.0625	3.37	2.58
6	139.0625	143.75	141.4063	1.66	0.93
7	141.4063	143.75	142.5781	0.82	0.11
8	142.5781	143.75	143.1641	0.41	0.30

Thus after eight iterations $|\varepsilon_a|$ finally falls below $\varepsilon_s = 0.5\%$, and the computation can be terminated.

Bisection Method

Errors for the bisection method. True and approximate errors are plotted versus the number of iterations.



$$|\varepsilon_a| = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| 100\%$$

provides an upper bound on the true error

$|\varepsilon_a|$ is always greater than $|\varepsilon_t|$

when $|\varepsilon_a|$ falls below ε_s, the computation could be terminated with confidence that the root is known to be at least as accurate as the prespecified acceptable level.

Bisection Method

- The number of iterations required to attain an absolute error can be computed a priori (before starting the iterations).
- Before starting the absolute error is less than: $E_a^0 = x_u^0 - x_l^0 = \Delta x^0$
- After the first iteration the error is less than: $E_a^1 = \frac{\Delta x^0}{2}$

- Since each iteration halves the error, a general formula relating the error and the number of iterations is:

$$E_a^n = \frac{\Delta x^0}{2^n}$$

$$n = \frac{\log(\Delta x^0 / E_{a,d})}{\log 2} = \log_2\left(\frac{\Delta x^0}{E_{a,d}}\right)$$

MATLAB M-file: bisect

```
function [root,ea,iter]=bisect(func,xl,xu,es,maxit,varargin)
% bisect: root location zeroes
%   [root,ea,iter]=bisect(func,xl,xu,es,maxit,p1,p2,...):
%      uses bisection method to find the root of func
% input:
%   func = function handle
%   xl, xu = lower and upper guesses
%   es = desired relative error (default = 0.0001%)
%   maxit = maximum allowable iterations (default = 50)
%   p1,p2,... = additional parameters used by func
% output:
%   root = real root
%   ea = approximate relative error (%)
%   iter = number of iterations

if nargin<3,error('at least 3 input arguments required'),end
test = func(xl,varargin{:})*func(xu,varargin{:});
if test>0,error('no sign change'),end
if nargin<4||isempty(es), es=0.0001;end
if nargin<5||isempty(maxit), maxit=50;end
iter = 0; xr = xl;
while (1)
  xrold = xr;
  xr = (xl + xu)/2;
  iter = iter + 1;
  if xr ~= 0,ea = abs((xr - xrold)/xr) * 100;end
  test = func(xl,varargin{:})*func(xr,varargin{:});
  if test < 0
    xu = xr;
  elseif test > 0
    xl = xr;
  else
    ea = 0;
  end
  if ea <= es || iter >= maxit,break,end
end
root = xr;
```

Bisection Method

- Find the root of $$f(m) = \sqrt{\frac{9.81m}{0.25}}\tanh\left(\sqrt{\frac{9.81(0.25)}{m}}4\right) - 36$$

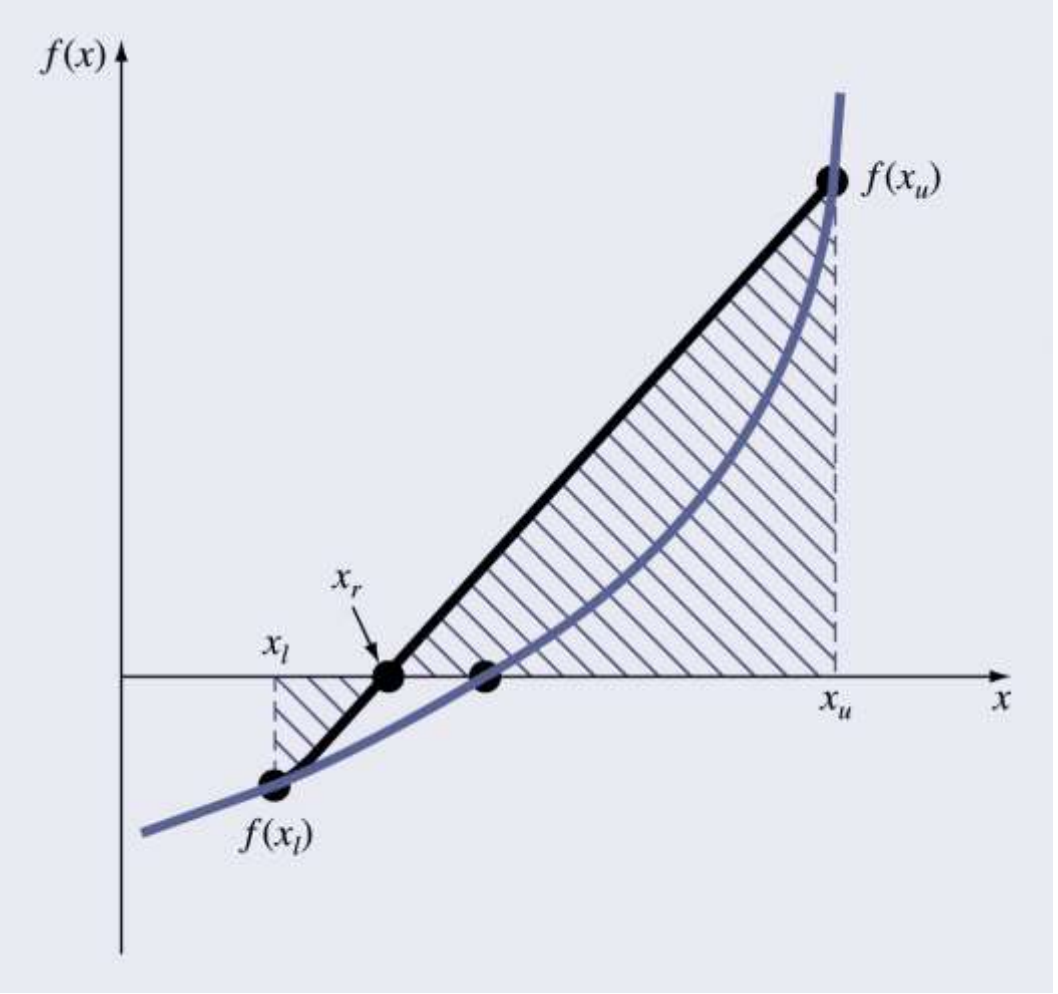
```
>> fm=@(m) sqrt(9.81*m/0.25)*tanh(sqrt(9.81*0.25/m)*4)-36;
>> [mass fx ea iter]=bisect(fm,40,200)

mass =
       142.74
fx =
  4.6089e-007
ea =
   5.345e-005
iter =
    21
```

False Position Method

False Position Method

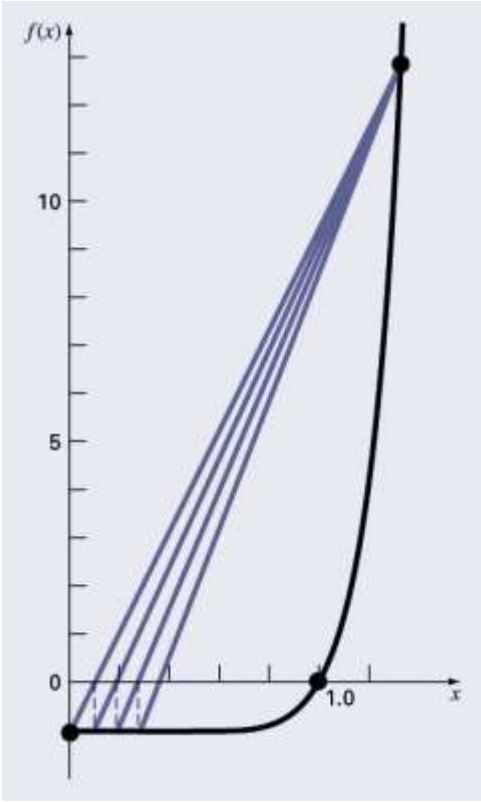
- The *false position* method is another bracketing method.
- It determines the next guess not by splitting the bracket in half but by connecting the endpoints with a straight line and determining the location of the intercept of the straight line (x_r).
- The value of x_r then replaces whichever of the two initial guesses yields a function value with the same sign as $f(x_r)$.
- False position formula $$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$$



Bisection vs. False Position

- Bisection does not take into account the shape of the function; this can be good or bad depending on the function!
- Slow convergence:

$$f(x) = x^{10} - 1$$



Example

Problem Statement. Use bisection and false position to locate the root of

$$f(x) = x^{10} - 1$$

Solution. Using bisection, the results can be summarized as

Iteration	x_l	x_u	x_r	ε_a (%)	ε_t (%)
1	0	1.3	0.65	100.0	35
2	0.65	1.3	0.975	33.3	2.5
3	0.975	1.3	1.1375	14.3	13.8
4	0.975	1.1375	1.05625	7.7	5.6
5	0.975	1.05625	1.015625	4.0	1.6

Thus, after five iterations, the true error is reduced to less than 2%. For false position, a very different outcome is obtained:

Example

Iteration	x_l	x_u	x_r	ε_a (%)	ε_t (%)
1	0	1.3	0.09430		90.6
2	0.09430	1.3	0.18176	48.1	81.8
3	0.18176	1.3	0.26287	30.9	73.7
4	0.26287	1.3	0.33811	22.3	66.2
5	0.33811	1.3	0.40788	17.1	59.2

After five iterations, the true error has only been reduced to about 59%. Insight into

This function violates the assumption that if f(x_l) is much closer to zero than f(x_u), then the root should be much closer to x_l than x_r.

Summary

- Although false position is generally superior to bisection, there are cases where this is not true.
- False position is one-sided: one of the bracketing points will stay fixed. This can lead to poor convergence.
- The results should always be checked by substituting the root estimate into the original equation and confirming that the result is close to zero.

Overview

- Roots
  - Points of a nonlinear equation where $f(x) = 0$
  - **Bracketing methods**: start with a guess for a range containing the root and reduce the width of the bracket iteratively. (reliable)
    - Bisection method
    - False position method
  - **Open Methods** (fast)
    - Fixed-point iteration
    - Newton-Raphson
    - Secant methods
  - Hybrid Methods: Brent's root finding method (Fast and reliable)
  - MATLAB: fzero function